Digital Systems

Lecture 15 & 16

Scheduling Processes & Messages

Mark van der Wilk

Hilary Term 2025

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

C Pointer Syntax

- Variable containing 1

  ```
  int a = 1;
  ```

-

-

-

C Pointer Syntax

- Variable containing 1

  ```
  int a = 1;
  ```

- Variable able to contain address to memory location containing `int`

  ```
  int* b;
  ```

-

-

C Pointer Syntax

- Variable containing 1

  ```
  int a = 1;
  ```

- Variable able to contain address to memory location containing `int`

  ```
  int* b;
  ```

- Set b to address of a

  ```
  b = &a
  ```

-

C Pointer Syntax

- Variable containing 1

  ```
  int a = 1;
  ```

- Variable able to contain address to memory location containing `int`

  ```
  int* b;
  ```

- Set b to address of a

  ```
  b = &a
  ```

- Set c to contents of a

  ```
  int c = *b
  ```

C Pointer Syntax for Structs

```
typedef struct dst {
  int feet;
  float inch;
} distance;
```

-
-
-

C Pointer Syntax for Structs

```
typedef struct dst {
  int feet;
  float inch;
} distance;
```

- Struct-typed variable

  `distance a; a.feet = 9001; a.inch = 11;`
-
-

C Pointer Syntax for Structs

```
typedef struct dst {
  int feet;
  float inch;
} distance;
```

- Struct-typed variable
  `distance a; a.feet = 9001; a.inch = 11;`
- Pointer to `distance`-typed variable, assign address of `a`
  `distance* b; b = &a;`
-

C Pointer Syntax for Structs

```
typedef struct dst {
  int feet;
  float inch;
} distance;
```

- Struct-typed variable
  `distance a; a.feet = 9001; a.inch = 11;`
- Pointer to `distance`-typed variable, assign address of `a`
  `distance* b; b = &a;`
- Content of address in b
  `(*b).inch == a.inch;  /* Or... */ b->inch == a.inch;`

Today

How are processes started?

What happens to the Operating System on startup?

Starting a Process

The first time a process runs, it is resumed just as if returning from a system call.

So we set up a fake exception frame that invokes the process body when resumed.

- `r0` = integer argument,
- `pc` = process body,
- `lr` = address of exit stub, in case body returns.

Creating “fake” Stack Frame

```
int start(...) {
    ...
    unsigned *sp = p->sp - FRAME_WORDS;
    memset(sp, 0, 4*FRAME_WORDS);
    // Macros for offsets in stack, ***_SAVE
    sp[PSR_SAVE] = INIT_PSR;  // Thumb bit switched on
    sp[PC_SAVE] = (unsigned) body & ~0x1; // Activate the proc
    sp[LR_SAVE] = (unsigned) exit; // Make it return to exit()
    sp[R0_SAVE] = (unsigned) arg;  // Pass arg in r0
    sp[ERV_SAVE] = MAGIC;
    p->sp = sp;
    ... }
```

Starting a Process

At this point, every process has a stack frame that:

-
-
-

Starting a Process

At this point, every process has a stack frame that:

- will restore the `pc` at the start of the process's function (i.e. it will run the function)
-
-

Starting a Process

At this point, every process has a stack frame that:

- will restore the `pc` at the start of the process's function (i.e. it will run the function)
- has values for all state and registers that respects the restoring convention (manual part + hardware part),
-

Starting a Process

At this point, every process has a stack frame that:

- will restore the `pc` at the start of the process's function (i.e. it will run the function)
- has values for all state and registers that respects the restoring convention (manual part + hardware part),
- has values in the registers that respect the calling convention of C functions (e.g. integer argument in `r0`).

Starting a Process

At this point, every process has a stack frame that:

- will restore the `pc` at the start of the process's function (i.e. it will run the function)
- has values for all state and registers that respects the restoring convention (manual part + hardware part),
- has values in the registers that respect the calling convention of C functions (e.g. integer argument in `r0`).

To start, just initiate context switch by returning from `system_call()`.

- Q: Return value?

Starting the OS

After `init()` called, all tasks have stack frames that will allow a context switch into them.

```
void __start(void) {
    /* Create idle process */
    ...
    /* Call the application's setup function */
    init();
    /* The main program morphs into the idle process. */
    os_current = idle_proc;
    set_stack(os_current->sp);
    idle_task();
}
```

Idle Process

```
void idle_task(void) {
  /* Pick a genuine process to run */
  yield();

  /* When there's nothing to do: */
  while (1) pause();
}
```

Where are we now?

We understand *how* tasks are switched and started.

-

-

Where are we now?

We understand *how* tasks are switched and started.

- OS is an interrupt that performs context switches (Q: remind me how)
-

Where are we now?

We understand *how* tasks are switched and started.

- OS is an interrupt that performs context switches
  (Q: remind me how)
- During context switch, interrupt stores some state on stack,
  assembly code stores the rest.

What is left to understand?

What is left to understand?

? How does the OS keep track of processes?

We have already seen that we need to start processes, and keep track of various process-specific quantities (e.g. stack location), in order to do context switches. How does the OS do this?

What is left to understand?

? How does the OS keep track of processes?

We have already seen that we need to start processes, and keep track of various process-specific quantities (e.g. stack location), in order to do context switches. How does the OS do this?

? How does the OS decide which process to run next?

What is left to understand?

? How does the OS keep track of processes?

We have already seen that we need to start processes, and keep track of various process-specific quantities (e.g. stack location), in order to do context switches. How does the OS do this?

? How does the OS decide which process to run next?

? How are messages transferred between processes?

And in particular, how is this done in interrupts?

? How does the OS keep track of processes?

We have already seen that we need to start processes, and keep track of various process-specific quantities (e.g. stack location), in order to do context switches. How does the OS do this?

Process Structure

```
struct _proc {
    int pid;            // Process ID
    char name[16];      // Name for debugging
    unsigned state;     // SENDING, RECEIVING, etc.
    unsigned *sp;       // Saved stack pointer
    int priority;       // Priority: 0 is highest
    proc waiting;       // Processes waiting to send
    int pending;        // Whether interrupt pending
    int msgtype;        // Message to send or receive
    message *msgbuf;    // Pointer to message buffer
    proc next;          // Next process in ready/send queue
};
```

Possible states

- `DEAD`: process that has exited. It is also the value present in slots of the table that have never been used.
- `ACTIVE`: process is ready to run. The process is either the current process, or the ready queue waiting to become current.
- `SENDING`: process is waiting to send to another; the process will appear in the sending queue of the destination.
- `RECEIVING`: process is waiting to receive a message; the `msgtype` field will specify who may send to it.
- `SENDREC`: call to `sendrec`, combining the functions of send and receive.
- `IDLING`: idle process, which runs when no other process is ready.

Process Structure

```
typedef struct _proc *proc;
...
static proc os_ptable[NPROCS];
```

-
-
-

Process Structure

```
typedef struct _proc *proc;
...
static proc os_ptable[NPROCS];
```

In `start(...)`:

```
   proc p = create_proc(name, roundup(stksize, 8));
```

•

•

•

Process Structure

```
typedef struct _proc *proc;
...
static proc os_ptable[NPROCS];
```

In `start(...)`:

```
   proc p = create_proc(name, roundup(stksize, 8));
```

In `create_proc(...)` we

-
-
-

Process Structure

```
typedef struct _proc *proc;
...
static proc os_ptable[NPROCS];
```

In `start(...)`:

```
   proc p = create_proc(name, roundup(stksize, 8));
```

In `create_proc(...)` we

- Initialise all the fields to sensible defaults / based on arguments.
-
-

Process Structure

```
typedef struct _proc *proc;
...
static proc os_ptable[NPROCS];
```

In `start(...)`:

```
   proc p = create_proc(name, roundup(stksize, 8));
```

In `create_proc(...)` we

- Initialise all the fields to sensible defaults / based on arguments.
- Find a memory location to initialise the stack (`sbrk(...)`)
-

Process Structure

```
typedef struct _proc *proc;
...
static proc os_ptable[NPROCS];
```

In `start(...)`:

```
   proc p = create_proc(name, roundup(stksize, 8));
```

In `create_proc(...)` we

- Initialise all the fields to sensible defaults / based on arguments.
- Find a memory location to initialise the stack (`sbrk(...)`)
- Find a memory location for proc variable (`new_proc(...)`)

Process Structure

```
typedef struct _proc *proc;
...
static proc os_ptable[NPROCS];
```

In `start(...)`:

```
   proc p = create_proc(name, roundup(stksize, 8));
```

In `create_proc(...)` we

- Initialise all the fields to sensible defaults / based on arguments.
- Find a memory location to initialise the stack (`sbrk(...)`)
- Find a memory location for proc variable (`new_proc(...)`)

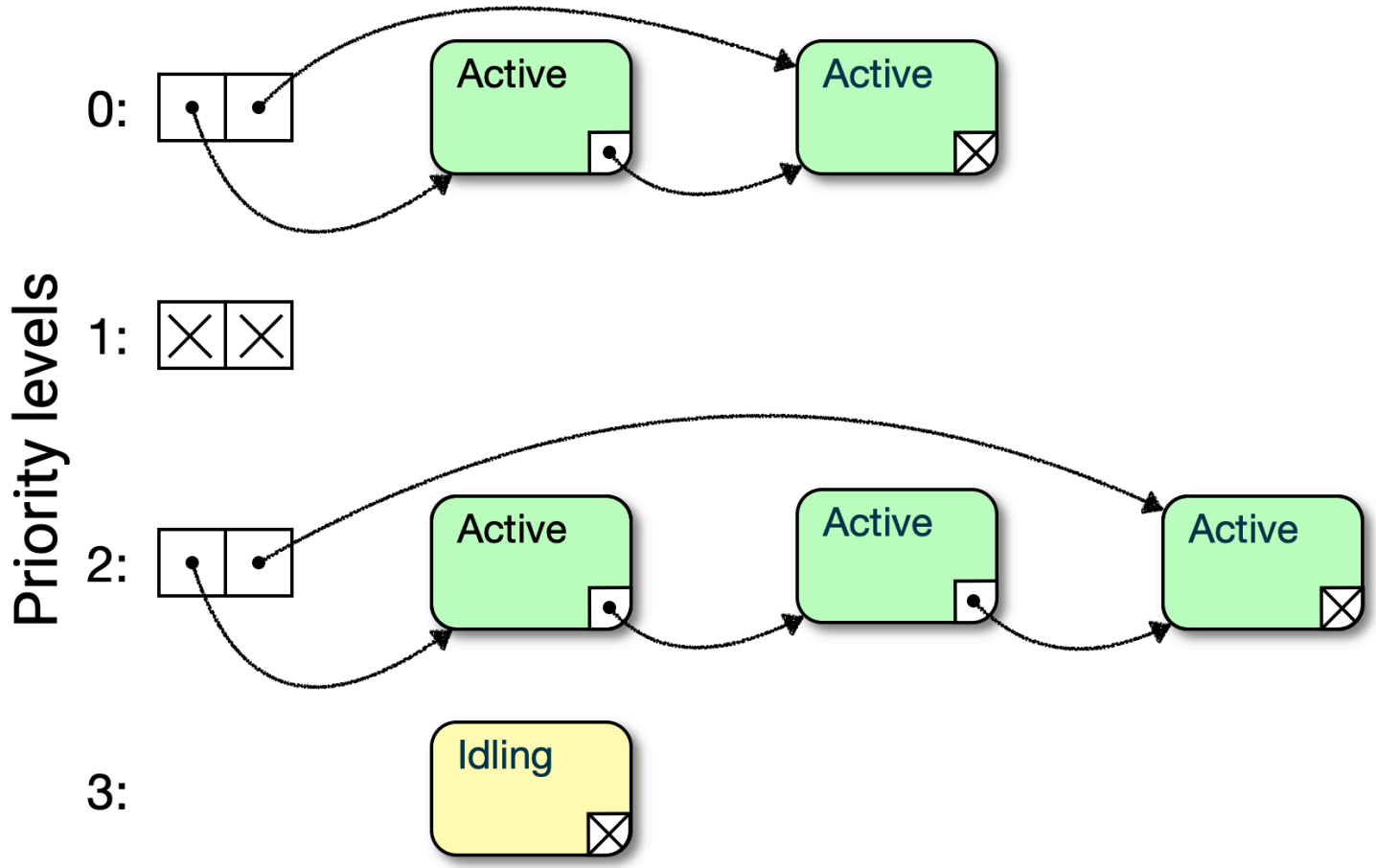
Draw memory layout (heap, stacks, ..., processes, main stack). ⟸

How does the OS decide which process to run next?

Start with the easy case: context switch initiated by `yield()`.

Ready Queues

In `ACTIVE` processes, the ready queue is implemented using the `next` field.



Ready Queues

```
#define NPRIO 3       /* Number of non-idle priorities */
...
static struct _queue {
    proc head, tail;
} os_readyq[NPRIO];
```

Ready Queues

```
#define NPRIO 3       /* Number of non-idle priorities */
...
static struct _queue {
    proc head, tail;
} os_readyq[NPRIO];
```

Ready Queues

```
#define NPRIO 3        /* Number of non-idle priorities */
...
static struct _queue {
    proc head, tail;
} os_readyq[NPRIO];
```

Last line of `start(...)`, after setting up fake exception frame...

```
    make_ready(p);
```

Adds process to appropriate queue.

Adding to the Queue: `make_ready()`

```
static inline void make_ready(proc p) {
    int prio = p->priority;
    if (prio == P_IDLE) return;

    p->state = ACTIVE;
    p->next = NULL;    // Because last in queue
    queue q = &os_readyq[prio];
    if (q->head == NULL)
        q->head = p;
    else
        q->tail->next = p;
    q->tail = p;                          }
```

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
-
-
-
-
-
-

•

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
- `microbian.c`: `__start()` is called.
-
-
-
-
-

•

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
- `microbian.c`: `__start()` is called.
- Calls `init()`, defined by “our” program.
-
-
-
-

•

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
- `microbian.c`: `__start()` is called.
- Calls `init()`, defined by “our” program.
- Sets up processes by calling `start()`.
-
-
-

•

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
- `microbian.c`: `__start()` is called.
- Calls `init()`, defined by “our” program.
- Sets up processes by calling `start()`.
- Creates handcrafted (rather than hardware) stack frame, so process can be started through normal return.
-
-

•

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
- `microbian.c`: `__start()` is called.
- Calls `init()`, defined by "our" program.
- Sets up processes by calling `start()`.
- Creates handcrafted (rather than hardware) stack frame, so process can be started through normal return.
- Calls `make_ready()` to add process to appropriate queue.
  At initialisation, all processes are `ACTIVE` and in the ready queue.
-

•

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
- `microbian.c`: `__start()` is called.
- Calls `init()`, defined by "our" program.
- Sets up processes by calling `start()`.
- Creates handcrafted (rather than hardware) stack frame, so process can be started through normal return.
- Calls `make_ready()` to add process to appropriate queue.
  At initialisation, all processes are `ACTIVE` and in the ready queue.
- Make current function the idle task.

•

Full Story of Startup

Have code side-by-side.

- `startup.c`: `__reset()` is called.
- `microbian.c`: `__start()` is called.
- Calls `init()`, defined by "our" program.
- Sets up processes by calling `start()`.
- Creates handcrafted (rather than hardware) stack frame, so process can be started through normal return.
- Calls `make_ready()` to add process to appropriate queue.
  At initialisation, all processes are `ACTIVE` and in the ready queue.
- Make current function the idle task.

- `yield()`, so OS starts scheduling procedure.

Selecting Next Process

We end up in `system_call(...)` again.

```
unsigned *system_call(unsigned *psp) {
    short *pc = (short *) psp[PC_SAVE]; /* Program counter */
    int op = pc[-1] & 0xff;  // Syscall number from SVC instr
    os_current->sp = psp;    // Save sp of the current process
    ...
    switch (op) {
    case SYS_YIELD:
        // Add current to back of Q. Choose next process.
        make_ready(os_current); choose_proc(); break;
        ... }
    return os_current->sp; }  // Return sp for next process
```

Selecting Next Process

Find highest priority process to run, remove it from queue, `return`.

```
static inline void choose_proc(void) {
    for (int p = 0; p < NPRIO; p++) {
        queue q = &os_readyq[p];
        if (q->head != NULL) {
            os_current = q->head;
            q->head = os_current->next;
            return;
        }
    }
    os_current = idle_proc;
}
```

? How does the OS decide which process to run next?

But this time, if we send a message.

? How does the OS decide which process to run next?

But this time, if we send a message.

Remember: Send and receive are also calls to OS!

? How does the OS decide which process to run next?

But this time, if we send a message.

Remember: Send and receive are also calls to OS!

`microbian.c`:

```
void NOINLINE send(int dest, message *msg) {
    syscall(SYS_SEND);
}
```

? How does the OS decide which process to run next?

But this time, if we send a message.

Remember: Send and receive are also calls to OS!

`microbian.c`:

```
void NOINLINE send(int dest, message *msg) {
    syscall(SYS_SEND);
}
```

? How does the OS decide which process to run next?

But this time, if we send a message.

Remember: Send and receive are also calls to OS!

`microbian.c`:

```
void NOINLINE send(int dest, message *msg) {
    syscall(SYS_SEND);
}
```

`hardware.h`:

```
#define syscall(op)     asm volatile ("svc %0" : : "i"(op))
```

System Call after `send()`

Just like `yield()`, `send()` simply causes a syscall interrupt.

```
unsigned *system_call(unsigned *psp) {
  short *pc = (short *) psp[PC_SAVE];
  int op = pc[-1] & 0xff;
  os_current->sp = psp;
  switch (op) {
  case SYS_SEND:
    mini_send(arg(0, int), arg(1, int), arg(2, message *));
    break; ...
  }
  return os_current->sp;
}
```

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

-
-

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

-
-

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

The destination process is `RECEIVING` and can receive message

-
-

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

The destination process is `RECEIVING` and can receive message

- Transfer message to destination. Destination `READY`. Add to queue.

-
-

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

The destination process is `RECEIVING` and can receive message

- Transfer message to destination. Destination `READY`. Add to queue.

-
-

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

The destination process is `RECEIVING` and can receive message

- Transfer message to destination. Destination `READY`. Add to queue.

The destination process cannot receive…

-
-

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

The destination process is `RECEIVING` and can receive message

- Transfer message to destination. Destination `READY`. Add to queue.

The destination process cannot receive…

- Multiple possible reasons for this (Q: which?)
-

Sending Messages

Remember the rules for communication by messages!

Two possible cases:

The destination process is `RECEIVING` and can receive message

- Transfer message to destination. Destination `READY`. Add to queue.

The destination process cannot receive…

- Multiple possible reasons for this (Q: which?)
- Must (somehow) keep sender in queue until process is ready to receive it.

Sending Queues

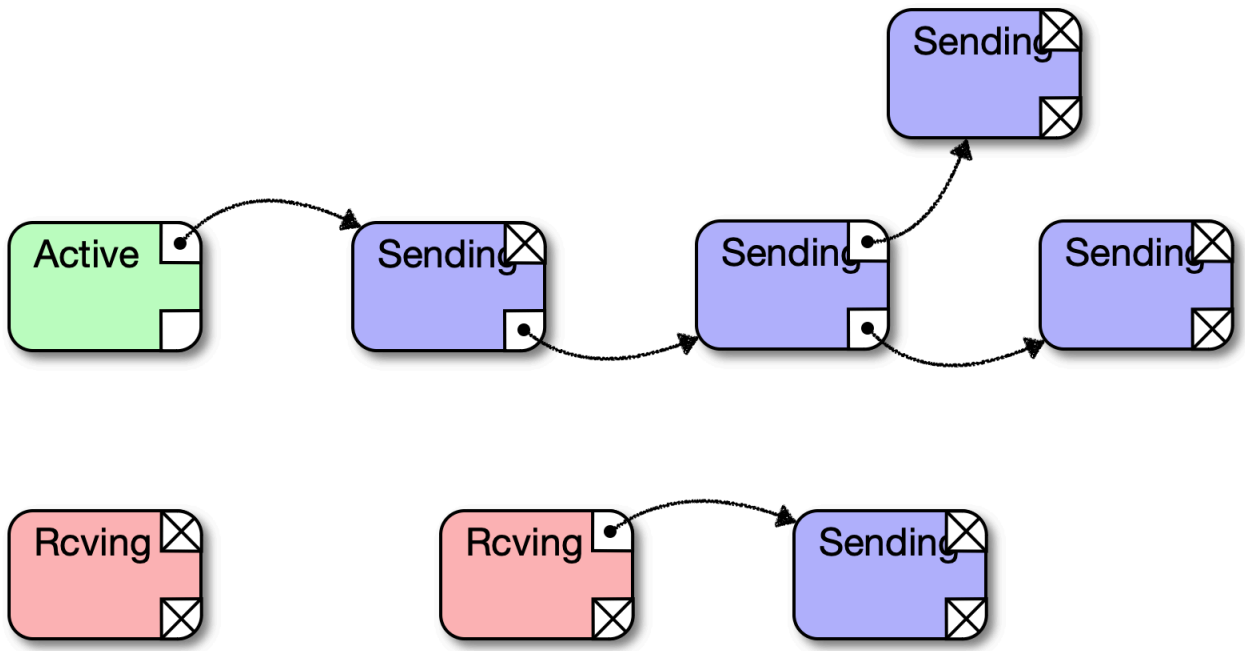
Processes can have multiple other processes waiting to send.

Next process trying to send is stored in `waiting`.

Rest of the send queue is chained through `next`.

Q: Does this conflict with the ready queue?

Each process has a queue of others waiting to send to it.



- Active process has 3 processes waiting to send to it.
- One process waiting to send also has a process waiting to send to *it*.
- One waiting to receive, but nothing to send to it yet.
- One waiting to receive, with one waiting to send to it, but not the right type of message.

System call after send()

```
static void mini_send(int dest, int type, message *msg) {
  int src = os_current->pid;
  proc pdest = os_ptable[dest];
  if (accept(pdest, type)) { // Receiver is waiting
    deliver(pdest->msgbuf, src, msg);
    make_ready(pdest); make_ready(os_current);  // *DISCUSS*
  } else {
    os_current->state = SENDING;
    queue_send(pdest);  // Sender waits. Join receiver's queue
  }
  choose_proc();
}
```

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

-
-

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

-
-

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

-
-

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

Suitable message is waiting to be delivered.

-
-

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

Suitable message is waiting to be delivered.

- Deliver message of suitable type, from process.
-

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

Suitable message is waiting to be delivered.

- Deliver message of suitable type, from process.
- Both processes can now continue.

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

Suitable message is waiting to be delivered.

- Deliver message of suitable type, from process.
- Both processes can now continue.

No suitable message can be delivered.

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

Suitable message is waiting to be delivered.

- Deliver message of suitable type, from process.
- Both processes can now continue.

No suitable message can be delivered.

-

? How does the OS decide which process to run next?

But this time, if we **receive** a message.

Three possible cases:

Can receive a message from an interrupt ⇒ discuss later.

Suitable message is waiting to be delivered.

- Deliver message of suitable type, from process.
- Both processes can now continue.

No suitable message can be delivered.

-

Receiving Messages

```
static void mini_receive(int type, message *msg) {
  if (/* interrupt is due */) { // later...
  } else {
    proc psrc = find_sender(os_current, type);// search send Q
    if (psrc != NULL) { // Is a sender waiting?
      deliver(msg, psrc->pid, psrc->msgbuf);
      make_ready(os_current); make_ready(psrc);
    } else { // No luck: we must wait
      set_state(os_current, RECEIVING, type, msg);
  } }
  choose_proc();
}
```

? How are messages transferred between processes?

And in particular, how is this done in interrupts?

Interrupt Handler

```
void default_handler(void) {
    int irq = active_irq(), task;
    if (irq < 0 || (task = os_handler[irq]) == 0)
        panic("Unexpected interrupt %d", irq);
    disable_irq(irq);
    interrupt(task);
}
```

Discuss code.

How are messages delivered

How are messages delivered

```
static inline void deliver(proc pdest, proc psrc)
{
    if (pdest->msgbuf) {
        *(pdest->msgbuf) = *(psrc->msgbuf);
        pdest->msgbuf->sender = psrc->pid;
    }
    make_ready(pdest);
}
```

Summary

- Process table
- Process states
- Ready queues
- Send queues
- Implementation of delivery schedule of messages

Notes contain another example of a device driver, and a discussion of `SENDREC`. Do take a look at this.

End of Term!

Next term, start from the complete bottom of the stack:

? How to we build a computer?

... starting from a transistor